Statistical Machine Learning - MSc Practical
Regularised Polynomial Regression

HT 2026

In this exercise, you will work through linear and polynomial regression. Our data consists of a training set $(x_i, y_i)_{i=1,\dots,n}$ and a test set $(\widetilde{x}_i, \widetilde{y}_i)_{i=1,\dots,n}$, where each dataset is composed of pairs of inputs/outputs (x_i, y_i), with $x_i \in \mathbb{R}$ and $y_i \in \mathbb{R}$, which are realisations of iid random variables distributed as (X, Y). Denote $R(h_\beta) = \mathbb{E}[(Y - h_\beta(X))^2]$ the risk under the squared loss function of a prediction rule h_β. Your goal is, for some (possibly nonlinear) input expansion $\phi : \mathbb{R} \to \mathbb{R}^{M+1}$, to learn a linear prediction rule of the form $h_\beta(x) = \phi(x)^\intercal \beta$ that has small generalisation risk.

Recall that the empirical risk minimiser is $h_{\widehat{\beta}}(x) = \phi(x)^\intercal \widehat{\beta}$ where

$$\widehat{\beta} = \arg\min_{\beta \in \mathbb{R}^2} J(\beta)$$

where

$$J(\beta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \phi(x_i)^\intercal \beta)^2 = \frac{1}{n} \| \mathbf{y} - \mathbf{\Phi} \beta \|^2$$

with

$$\mathbf{y} = (y_1, \dots, y_n)^\intercal \qquad \mathbf{\Phi} = \begin{pmatrix} \phi(x_1)^\intercal \\ \phi(x_2)^\intercal \\ \vdots \\ \phi(x_n)^\intercal \end{pmatrix}.$$

The following Python code imports the relevant libraries and loads the dataset.

```
# Import libraries
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn import model_selection
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error as mse
font = {'size'   : 18}
plt.rc('font', **font)

# Set the seed for reproducibility
np.random.seed(0)

# Import data
df_train = pd.read_csv('regression_train.csv')
X_train = df_train['x'].values.reshape(-1,1)
y_train = df_train['y'].values.reshape(-1,1)
df_test = pd.read_csv('regression_test.csv')
X_test = df_test['x'].values.reshape(-1,1)
y_test = df_test['y'].values.reshape(-1,1)
```

1 Visualisation

It is often useful to start with some exploratory data analysis. For this data set, you can use a scatter plot, since it only has two features (x and y).

- Visualise the training and test data. What do you observe? For example, can you make an educated guess on the effectiveness of simple linear regression in predicting the data?

2 Simple Linear Regression

We first consider simple linear regression, that is $\phi(x) = (1 \;\; x)^\intercal$ and

$$\mathbf{\Phi} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \\ 1 & x_n \end{pmatrix}$$

- Generate the matrix Φ for the training and test data, either directly or by using the built-in function `PolynomialFeatures`
- Using the function `LinearRegression`, fit the model on the training data, print the estimated parameter $\widehat{\beta}$, and the empirical risk of the learned prediction rule on the training and test sets
- Plot the training, test data and the fitted linear prediction rule on the same figure

2.1 Gradient Descent

One way to solve linear regression is through gradient descent (GD). Let η be the step size. The update at iteration t of the GD algorithm is of the form

$$\beta^{(t)} = \beta^{(t-1)} - \frac{2\eta}{n}\mathbf{\Phi}^\intercal(\mathbf{\Phi}\beta^{(t-1)} - \mathbf{y}).$$

Let T be the total number of iterations.

- Create a function `GD` that takes as inputs $\mathbf{\Phi}$, $\mathbf{y}$, T and η and returns $\beta^{(t)}$ and $J(\beta^{(t)})$ at each iteration $t = 0, \ldots, T-1$
- Use this function with a step-size of 0.005. Plot the value of the objective function as a function of the number of iterations. How many iterations does it take to converge? Try other values of the step-size. Does it help?
- To resolve the issue, we standardise the data so that the sample mean is zero and the sample variance is 1. Use the `StandardScaler` function to do so. Rerun GD with a step-size of 0.1. Plot the value of the objective function as a function of the number of iterations. Rerun the GD algorithm with $\eta = 1$ and $\eta = 0.01$. What do you observe?

2.2 Stochastic Gradient Descent

Another way to solve linear regression is through stochastic gradient descent (SGD). We consider here SGD with a mini-batch of size 1. The update at iteration t of the SGD algorithm is of the form

$$\beta^{(t)} = \beta^{(t-1)} - 2\eta_t\phi(x^{(t)})(\phi(x^{(t)})^\intercal\beta^{(t-1)} - y^{(t)})$$

where $(x^{(t)}, y^{(t)})$ is picked uniformly from the training data. We take $\eta_t = \frac{0.1}{t^{0.9}}$.

- Create a function `SGD` that takes as inputs $\mathbf{\Phi}$, $\mathbf{y}$, T and a vector η and returns $\beta^{(t)}$ and $J(\beta^{(t)})$ at each iteration $t = 0, \ldots, T-1$
- Use this function to estimate the parameter β using SGD. Report the value of the objective function as a function of the number of iterations.

3 Polynomial linear regression

Consider a polynomial linear regression model of order M, where

$$\phi(x) = (1 \quad x \quad x^2 \quad \dots \quad x^M)^\intercal.$$

- For $M = 0, \dots, 9$, estimate the parameter β using empirical risk minimisation. Plot the training and test risk as a function of the degree of the polynomial.
- Plot the learned prediction rule that has the lowest test risk. Discuss which values of M lead to overfitting/underfitting.

4 Ridge regression

We now consider the class of polynomials of order $M = 12$, with the addition of an L2 penalisation term. The (ridge regression) objective function is now

$$J(\beta) = \frac{1}{n}\|\mathbf{y} - \mathbf{\Phi}\beta\|^2 + \frac{\lambda}{n}\|\beta\|^2$$

where we take

$$\phi(x) = \left(1 \quad \frac{x-\mu_1}{\sigma_1} \quad \frac{x^2-\mu_2}{\sigma_2} \quad \dots \quad \frac{x^M-\mu_M}{\sigma_M}\right)^\intercal$$

with $\mu_j = \frac{1}{n}\sum_{i=1}^n x_i^j$ and $\sigma_j^2 = \frac{1}{n-1}\sum_{i=1}^n (x_i^j - \mu_j)^2$.

- Why should we standardise the polynomial inputs here?
- Apply ridge regression with a range of values for the regularisation parameter λ. Plot the values of the training and test risks as a function of λ (use a logarithmic scale for the x axis (λ)).
- Plot on the same figure the training, test data, and the fitted prediction rule that minimises the test risk.

5 Over-parameterisation and double descent

Assume now that $x \in [7, 13]$, and consider the feature expansion

$$\phi(x) = \left(P_0\left(\frac{x-10}{3}\right), P_1\left(\frac{x-10}{3}\right), \dots, P_M\left(\frac{x-10}{3}\right)\right)$$

where $P_m(x)$ is the mth Legendre polynomial on $[-1, 1]$. Using such features would avoid scaling issues when dealing with large values of M.

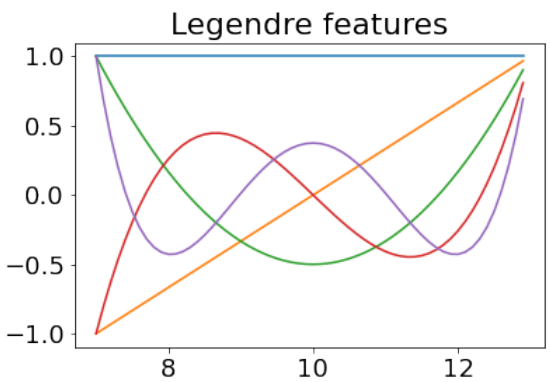

Figure 1: Legendre functions $P_m(\frac{x-10}{3})$ for $m = 0, \dots, 4$.

The following function returns the n-by-$(M+1)$ Φ matrix of Legendre features from the vector of n inputs $(x_1, \dots, x_n)$.

```python
# Function to obtain Legendre feature matrix Phi
def legendre_features(X, M):
    legend_reg_transform = lambda x: np.polynomial.legendre.legvander((x-10)/3, deg=M)
    return legend_reg_transform(X.reshape(-1,1)).squeeze()
```

- For values of M both smaller and larger than the sample size n, estimate the parameter β using empirical risk minimisation (with no regularisation). In the over-parameterised regime, use the minimum-norm interpolator (this is what is implemented by default in scikit-learn). Plot the training and test risk as a function of the degree of the Legendre features. Plot also the norm of the estimated parameter as a function of the degree.
- Discuss the shape of the training/test risk curves; identify where the minimum is achieved.
- Plot the learned prediction rule for $M = 10000$; compare to the estimated prediction rule in part 3.